

Interview Prep — In-Depth Reference

Everything about the pipeline, in enough depth to answer any follow-up. Organized by pipeline stage, then by the war stories/bugs, then by the numbers, then rapid-fire Q&A. Read once fully before the interview; use as a lookup during prep, not something to read from live.

0. The 30-second answer, memorize this verbatim

"Given a raster circuit drawing, the pipeline detects symbols with a YOLO model trained entirely on self-generated synthetic data, classifies each one with a five-source evidence-fusion system that never trusts a single signal, resolves instance identity and values with OCR, traces wire connectivity with classical computer vision, and emits a JSON netlist plus a graph — matching the brief's exact schema. No stage was trained or tuned against manually labeled real drawings; two real drawings exist only as held-out, hand-verified evaluation ground truth."

1. Preprocessing

What it does: grayscale conversion → adaptive binarization → deskew → speckle denoise → optional blue/black color-layer split.

Binarization — `binarize_block_size=31` , `binarize_c=7` . Adaptive (local-mean) thresholding, not a single global threshold, because scan/ render quality can vary across one drawing. The block size (31px neighborhood) and constant (7, subtracted from the local mean) were tuned by visually comparing the reconstructed binary image against the source until strokes matched exactly, not guessed.

Speckle removal — `denoise_min_speckle_area=2` . Connected-component filtering, not a morphological open. *Why not open:* a 2×2 erosion kernel erases any 1px-wide stroke — and this drawing's actual line art is 1px wide throughout, so a naive open would destroy most of the content, not just noise. Component-area filtering removes literal isolated specks (area ≤ 2 px) without touching real strokes, because it only looks at how big a blob is, not how thin it is.

Deskew — `deskew_max_angle_deg=5` , via `cv2.minAreaRect` . Small- angle correction only. These are digitally-generated line art drawings, not scanned pages — large rotations are out of scope by design, and `estimate_skew_deg` normalizes `minAreaRect` 's angle output into a small correction around 0 rather than snapping to the nearest 90°.

Color-layer split (D5 only). D5 draws some conductors in blue, others in black. Splitting channels is nearly free and gives a strong net-membership prior — but it's used only as a cross-check that raises or lowers confidence in the connectivity stage, never a hard dependency, because D4 is monochrome and any stage that *required* color would fail on half the input.

Why OCR and junction-dot detection both run *before* localization, not after: a text glyph has as many skeleton corners/endpoints as a real symbol and would otherwise become a false localization candidate.

Detecting text tokens and junction dots first lets the localizer exclude those regions. This ordering decision is easy to miss and worth stating explicitly if asked “walk me through the exact stage order.”

2. Localization — YOLOv8n

What it does: proposes candidate symbol bounding boxes.

Why YOLOv8n specifically: small (3M params), fast enough to run on a 4GB laptop GPU (or CPU) in milliseconds per image, and — critically — trainable entirely on self-generated labels, so it never required manual bounding-box annotation.

How training data was generated, in detail (`generate_synthetic_dataset.py`): KiCad-rendered reference symbols (see §7 below) are pasted onto blank canvases at the *real drawings*’ size distribution (25–90px on a side — measured from D4/D5, not guessed), rotated 0/90/180/270°, with synthetic Manhattan-routed wire clutter and small dash-cluster “text” clutter added as *unlabeled* negatives (so the detector learns not to fire on wires or text, without ever being told “this is not a symbol” explicitly — it just never sees a label there). The bounding-box label is a direct byproduct of where the generator chose to place each symbol — nobody looked at an image and drew a box.

The actual training mix that shipped (`symbol_detector_mixed`): three synthetic generators combined — 500 “scatter” images (sparse random placement, `generate_synthetic_dataset.py`), 300 “dense” images (14–26 symbols/canvas, same script with tighter packing params), and 150 “ladder” images (`generate_ladder_circuits.py` — a *topologically realistic* generator: closed rectangular border, top/bottom rails, vertical branches with 1–2 components in series, junction dots at rail intersections — built specifically because scattered-random placement teaches “what a symbol looks like” but nothing about real layout structure). 950 total training images, 60 held-out synthetic validation images, 21 classes.

Why a topology-realistic generator was built as a *third*, separate generator instead of just making the scatter one denser: it isolates two different questions that a single “harder scatter” test would conflate — “does the model handle realistic circuit topology” vs. “does it generalize to unfamiliar icon styles.” Mixing both into one dataset made a low score uninterpretable; separating them made each failure attributable to a specific cause.

Confidence threshold — `yolo_conf_threshold=0.12` (lowered from an initial 0.25). Verified directly by sweeping 0.25/0.1/0.05 against the raw detector on D5: detections in the 0.1–0.25 band were overwhelmingly real components, not noise, meaning the higher default was a *recall* bottleneck, not a precision safeguard. This is a good example of “measured, not assumed” — the instinct would be that a higher threshold is always safer.

Why YOLO’s own classification output is not trusted as final: it’s trained purely on synthetic composites. Its *localization* is sound because “is there a symbol-shaped blob here” transfers well from synthetic to real; its *classification* transfers less reliably because fine-grained class distinctions (NPN vs. PNP, diode vs. zener) are exactly where synthetic-to-real domain gap bites hardest. So YOLO’s class + confidence is carried through as *one vote* into the fusion system (§3), not the final answer.

The classical fallback that still exists: a density-based localizer (`localize/proposals.py`) — connected-component analysis of skeleton branch/endpoint density, no training data needed at all — is the original, first-built approach, and the pipeline automatically falls back to it if no trained YOLO weights are present. *Why keep it rather than delete it once YOLO shipped:* “always runnable end to end, even before a model is trained” was a deliberate discipline from day one (the “day-1 walking skeleton” principle threaded through the whole codebase — an empty reference library, a missing probe artifact, and a missing YOLO checkpoint all degrade gracefully instead of crashing).

Measured, honest limitation: a manual audit of the *original* density-based localizer found a ~60% false-positive rate on D4 (almost entirely wire bends and rail junctions misread as symbols) — this is *why* YOLO replaced it as the default, not a hypothetical.

Two retrain attempts that regressed and were reverted (full detail in §9 — know these numbers cold):

- Expanding to 21 classes: synthetic mAP50 >0.98 on new classes, real detection F1 0.914→0.686.
- Mixing in a real-world third-party dataset: synthetic mAP50 0.912 (fine), real detection F1 0.879→0.174.

3. Classification — five-source evidence fusion

This is the architecturally deepest part of the system and the most likely to get drilled on. Know the exact formula and every source.

3.1 The core idea and the fusion math

Each source produces a `ClassificationEvidence(source, class_scores, confidence, metadata)` — a dict of `{class_name: score}` plus one scalar confidence for *this particular reading*, or an explicit “no evidence” if the source has nothing useful to say (never a forced guess).

Fusion sums, per class: `total[class] += weight[source] × evidence.confidence × evidence.class_scores[class]` across every source that fired, then takes the argmax. Two things this buys over a fixed priority order (“OCR beats vision beats YOLO”):

1. A source’s *own* certainty about *this specific reading* matters, not just a fixed per-source weight — a garbled OCR read (low confidence) contributes almost nothing even though OCR as a source has a real, nonzero base weight.
2. Two sources agreeing pushes a class’s total score up additively — agreement is rewarded structurally, not via a special-cased rule.

Default weights (config-overridable, `evidence_weights` in `config/default.yaml`): YOLO 1.2, DINOv2 1.0, linear probe 1.0, OCR text prior 1.0, geometry specialists 1.0. *Why YOLO gets a modest edge:* it’s the supervised, in-domain signal (trained on our own schematic symbols); DINOv2 is self-supervised on natural photographs and has never seen a schematic. Not a hard override — a confident DINOv2/probe/OCR reading can still outvote a weak YOLO one, and does, regularly.

3.2 Source 1 — YOLO's own class head

Already covered in §2. Votes `{yolo_class: 1.0}` at `confidence = yolo_confidence` (the detector's own softmax-ish score for that box). Never abstains (always has *some* class prediction) — but a very low `yolo_confidence` means it contributes almost nothing regardless.

3.3 Source 2 — DINOv2 nearest-neighbor match

What DINOv2 is: a self-supervised Vision Transformer (Meta AI) — `dinov2_vitb14`, the “base” variant, 768-dim embeddings. Each candidate crop is embedded, then matched via cosine similarity (FAISS `IndexFlatIP` — inner product on L2-normalized vectors = cosine similarity) against a reference library of embedded KiCad-sourced symbol images.

Why DINOv2 over CLIP: DINOv2 is trained for dense visual similarity — better for near-duplicate/shape matching of technical line art. CLIP is trained for image-text alignment and is measurably weaker at fine geometric distinctions between visually similar symbols (a resistor's zigzag vs. an inductor's coil are “similar enough” in CLIP's image-text-aligned space in a way they aren't in DINOv2's shape space).

Why DINOv2-base (768-dim) over DINOv2-small (384-dim): better separation of visually similar symbols, ~2–3× the per-image compute cost on CPU — affordable since classification was never the pipeline's throughput bottleneck.

Why nearest-neighbor over a fine-tuned classifier: this is the single decision that makes the system “generic” in the sense the brief asks for. Adding a new symbol class is *dropping in reference crops and rebuilding an index* — no retraining, no new labeled dataset, no GPU time. A fine-tuned classifier would need to be retrained (with the regression risk demonstrated twice in §9) every time the class set changes.

Rotation/mirror augmentation. DINOv2 embeddings are not rotation-invariant — a sideways resistor lands far from its upright reference in embedding space. The library is augmented at build time with 0/90/180/270° rotations plus a horizontal mirror (8× per source image), and *the matched rotation is retained* — it's what orients the terminal template onto the candidate bbox later (§5). This was a deliberate choice over rotation-invariant descriptors (Hu moments) or axis-canonicalization, both rejected because they're measurably weaker at separating visually similar symbols, and axis canonicalization is unstable for near-square symbols.

`unknown_similarity_threshold=0.62`. Below this, DINOv2 abstains entirely rather than forcing a bad match through — a source with “some” similarity but not enough is worse than no evidence at all, since it would otherwise vote confidently for a wrong class.

`match_topk`, **not just top-1**. Returns the top-3 *distinct classes* (collapsing the many augmented variants of one class down to its best score), over-fetching 20 raw neighbors first so the top few hits being all-one-class's rotations doesn't starve out genuinely different candidates. This top-k list feeds the ambiguity-routing logic for geometry specialists (§3.6) and is exposed on every output `Symbol` as `top_k` + `margin` for explainability.

Embedding library caching (a real performance bug, fixed). `ReferenceLibrary.build()` originally re-embedded every reference crop through DINOv2 on *every single pipeline run* — fine at ~184 entries, measured at 45–85 seconds once the library grew to 288 after adding style variants. Fixed with a

signature-keyed cache (`<reference_dir>/index.npz` , signature = hash of every source file's path/mtime/size + the embedding model name + rotation config) that auto-invalidates on any real change and cut classify-stage time from ~46s to ~7s on a cache hit — verified both the speedup *and* that touching a reference file correctly triggered a rebuild.

3.4 Source 3 — the linear probe

What it is: a scikit-learn `LogisticRegression` trained on frozen DINOv2 embeddings of domain-randomized synthetic crops — a *learned decision boundary* over the same embedding space DINOv2's nearest-neighbor match uses, tested as an independent hypothesis: does a learned boundary generalize better than "closest single reference point"?

Why built as a genuinely separate evidence source, not a replacement for nearest-neighbor: if the probe turns out worse on some case, it can only add a wrong vote that DINOv2's own correct vote outweighs — it can't override a good source, it can only supplement. This mirrors the project's whole "never let one signal be the whole answer" philosophy applied at the sub-classifier level, not just across YOLO/DINOv2/OCR.

Domain randomization (`scripts/domain_randomize.py`). Every training crop gets: rotation jitter $\pm 4^\circ$ (small — a real orientation change is handled separately by the 0/90/180/270 augmentation; this simulates imprecise drafting, not a different orientation), stroke-width dilate/erode (different rendering tools produce different line thickness), Gaussian blur, sensor noise, contrast/gamma jitter, and a simulated JPEG re-encode. Explicitly *not* included: non-uniform aspect distortion (would change relative proportions some geometry specialists key off) or rotation past a few degrees. Every transform was visually spot-checked on a sample grid before being trusted.

Training scale: every reference image $\times$ 4 rotations $\times$ mirror $\times$ 40 jittered variants $\approx$ 11,500 crops, embedded in ~230s on GPU, logistic regression fit in ~1s. Synthetic held-out validation accuracy ~99% — *explicitly logged in the training script's own output as not evidence of real performance* — only `edp eval` against the hand-verified golden set answers that question.

3.5 Source 4 — OCR reference-designator / part-number prior

The core idea: engineering reference designators are a documented convention (IEC 61346 / ANSI Y32.2) — `R` for resistor, `C` for capacitor, `ZD` for zener, etc. — and specific part numbers are even stronger evidence (`BC547` is unambiguously an NPN transistor). This is knowledge no vision model has access to, and it's completely deterministic once OCR produces usable text.

Three confidence tiers, gated by specificity, not treated as one flat signal:

- **Exact part number** (`BC548` , `1N4007` , `MCT2E`) $\rightarrow$ confidence 0.95, near-authoritative.
- **Clean designator** (`R6` , `ZD1`) $\rightarrow$ confidence 0.55, a moderate vote — enough to matter, not enough to force a wrong answer through alone.
- **Unparseable/ambiguous** $\rightarrow$ contributes *nothing at all*, not weak evidence. This was a deliberate design rule stated up front: never let garbled OCR outvote confident visual evidence by treating noise as a weak-but-nonzero signal.

The table is declarative, not hardcoded in the inference path (`config/reference_designators.yaml`) — adding a new designator or part family is a YAML edit, consistent with the “add capability via data, not code” principle running through the whole project. Some designators are deliberately *absent*: `T` is not in the table at all, because in this project’s own two evaluation drawings it means Transformer in one (D5’s T1) and Transistor in the other (D4’s T1/T2/T3) — a forced prior there would be wrong roughly half the time by the very convention it’s meant to encode, so it’s left to visual evidence and part-number matching alone. Same reasoning for `U` (generic IC — covers far more real classes than this library models).

Where an LLM helped, and exactly how: authoring this table once, at build time — not called at runtime. The shipped system queries a static YAML file on every request, never a model. If asked “did you use an LLM,” this is the honest, specific answer: yes, as a documentation/ authoring aid, never as a runtime dependency.

A part-number match can win even when the designator alone wouldn’t: checked *before* designator matching, and matched via substring search, not anchored to the string’s start — so `"T2BC547"` (a designator `T2` concatenated with a part number, common OCR behavior on tightly-spaced labels) correctly resolves to `BJT_NPN` via the `BC547` substring match, even though `T` itself has no table entry.

3.6 Source 5 — geometry specialists

When invoked, and why not always: only when fusion’s own current winner (or ≥ 2 of the top-3 candidates) falls inside a *known* confusion group — never on every symbol. This keeps the fast, deterministic path the default; geometry is the slowest source (a handful of OpenCV calls) and only useful when there’s a real, specific ambiguity to resolve.

Battery vs. Capacitor vs. Capacitor_Polarized — the group with the most real confusions. Detects “plates” as connected components with a wide ($\geq 14\text{px}$), non-tall ($< 50\%$ of crop height), low-fill-ratio bounding box (a thin swept stroke fills far less of its box than a filled shape — this is what lets it find *curved* plates that a simple row-scan can’t). Decision rule: ≥ 4 plates $\rightarrow$ Battery (multi-cell convention). Exactly 2 plates, one curved $\rightarrow$ Capacitor_Polarized (this project’s actual drawn convention for polarity — different from the reference library’s own KiCad rendering, which encodes polarity via plate *thickness* instead; both conventions are real and the library now covers both). 2 flat plates + a detected “+” mark (found via a small, roughly-square connected component with ink concentrated along its middle row *and* column but not its corners) $\rightarrow$ Battery (single-cell style). 2 flat plates, no “+” $\rightarrow$ Capacitor.

Real bug caught during development: an early plate-detector used “row spans a wide fraction of total crop width” — broke immediately on a real candidate box that also contained adjacent value-label text (`"C1 10uF 50V"`) sharing the row, diluting the width fraction below threshold. Rewritten to use per-row *longest contiguous run length* (text glyphs and plates don’t merge into one run even on the same row), which is immune to unrelated text elsewhere in the same row.

BJT (either polarity) vs. Potentiometer. A BJT is drawn inside a circle with three leads radiating from its boundary; a potentiometer is a rectangular resistor body with a wiper arrow entering from the side — no circle anywhere. Circle presence, via `cv2.HoughCircles`, is the primary and much more reliable signal; rectangle-body detection (contour fill-ratio + aspect ratio) only fires as confirmation when no circle was found.

Real bug caught: the default Hough accumulator threshold (`param2=15`) false-fired a circle on the project's own Potentiometer reference render — tuned to 30 empirically, verified this rejects that false positive while still finding every real BJT circle tested (3 real crops + 2 reference renders).

NPN vs. PNP — built, tested, deliberately NOT shipped. Emitter- arrowhead direction (points away from the circle center = NPN, toward = PNP), found via triangular-contour detection on the arrowhead. Measured 2/2 correct on the project's own clean reference renders, but only 1/3 correct on real D4 transistor crops — worse than chance there. Left in the codebase (unit-tested, documented) but excluded from the routing table that decides which specialist to call. *This is one of the strongest "why should I trust your engineering judgment" answers available* — it demonstrates the same measure-before-trusting discipline applied even to work that was fully built and worked in principle.

4. Text association (OCR)

Tesseract, not a learned scene-text model, and why: deterministic, dependency-light, adequate on clean synthetic-quality glyphs once upscaled. The OCR module is isolated behind a narrow interface specifically so a model like PaddleOCR could be swapped in if measured results ever justified it — a documented, not-yet-needed escape hatch.

Upscaling — `upscale_factor=3` , Lanczos interpolation. D5's labels are a few pixels tall at native resolution; Tesseract has an effective floor around ~20px of glyph height, below which output is noise regardless of preprocessing. This is the single highest-impact OCR decision. *Honest caveat:* upscaling reconstructs, it does not add information — a genuinely sub-5px glyph stays unreliable no matter the upscale factor, and this is stated as a real limitation, not papered over.

Multi-orientation passes — `[0, 90, 270]` . D4 sets its "LOAD" label vertically; a single horizontal OCR pass would silently drop rotated labels rather than failing loudly.

Multi-config merge — the current, general-purpose fix (not D4/D5-specific). Runs *three* Tesseract PSM configurations per orientation (`--psm 11` , `--psm 12` , and `--psm 6` with the dictionary disabled) and keeps every reading from every pass, deliberately without deduplicating. *Why measured this way:* swept every PSM×upscale combination against known designators on *both* drawings first — no single config won on both (PSM 12 helped D4, PSM 6-no-dictionary helped D5 more) — merging all three strictly beat every individual config on both. *Why no deduplication, given a first attempt did dedupe:* keeping only the highest-confidence reading per overlapping region *lost* a real D4 answer — a correct `"$1"→"S1"` switch-designator reading was crowded out by a higher-confidence but *wrong* reading of the same region from a different PSM pass. Tesseract's confidence score for short, garbled, technical tokens isn't a reliable correctness signal, so picking a single "winner" by confidence was actively harmful; passing every reading through and letting downstream nearest-token matching naturally surface the correct one (redundantly, but harmlessly) measured better.

Downstream consequence of not deduplicating: the code that gathers "text near this symbol" (`nearby_token_text`) now returns up to 6 tokens (raised from 3) so duplicate readings of one label don't crowd out a second, genuinely different nearby token (e.g. a value string next to a designator).

A second, deeper bug this surfaced (worth walking through if asked “tell me about a subtle bug”): the OCR-designator prior originally normalized-and-matched the *whole joined multi-token string* as one blob, anchored at its start. One leading garbage token before a perfectly good reading (`"= ae R6 R6 R6 R6"` → normalized to `"=AER6R6R6R6"`) silently defeated an otherwise-correct `R6` match, because the regex requires the string to *start* with letters+digits. Fixed by checking each whitespace-separated token independently instead of the joined blob — a latent bug that existed before the multi-pass OCR change, just invisible until more OCR noise made it likely to actually trigger.

Safe OCR substitutions — narrow and justified, not general correction. `$` → `S` is the one entry in the table (`"S1"` — a switch designator — reliably OCR'd as `"$1"` , since `$` has no legitimate meaning in a component label). Deliberately *not* general character- level error correction, which would risk turning a low-confidence read into a confident-looking wrong one.

Instance ID assignment — global greedy matching, not per-symbol nearest-match. If done independently per symbol, two different nearby symbols can both claim the same OCR'd id (`"R4"` assigned twice), corrupting every downstream `connections` reference to that id. Fixed by collecting all (distance, symbol, id-text) candidates globally, sorting by distance, and greedily claiming each id-token and each symbol at most once.

5. Wire / connectivity extraction

Why classical CV, not a learned model: these drawings are thin, high-contrast, mostly-orthogonal strokes — the textbook case for skeletonization. No training data exists for a learned segmentation model and none is needed for this input distribution; a classical approach stays fully deterministic and traceable to a specific pixel rule, which matters for the same “must be able to justify every decision” reason threaded through the whole project.

Pipeline: subtract symbol bboxes from the binary image → skeletonize the remainder (`skimage.morphology.skeletonize` , morphological thinning to 1px width) → build a graph from the skeleton.

Junction-dot detection — local fill-ratio, not contour matching. A filled dot at a wire crossing means “connected”; a bare crossing means “not connected” — this is a real, load-bearing drawing convention, not a guess. *Why not* `cv2.findContours` *on the full binary*: a dot touching a wire (nearly every real dot) becomes part of one giant wire-shaped contour, not a small circle — an early version of this found only ~30 isolated dots total. Fixed by checking local ink *fill-ratio* within a small disk (`junction_dot_max_radius=5`) around each skeleton branch point instead — a dot fills most of a small disk around it regardless of what it's touching; a plain crossing doesn't. `junction_dot_fill_ratio = 0.6` was tuned by visual iteration against known dot/non-dot crossings.

Crossing decomposition — the single highest-leverage rule in this stage. Every degree-4 skeleton node (four wire segments meeting) is a decision point. With a confirmed dot: kept as one junction node, all four branches share a net. Without a dot: the node is *structurally split* into two independent pass-through paths (the horizontal pair and the vertical pair) *before* connected components are computed — so the two conductors land in genuinely different graph components. This encodes “crossing ≠ connection” into

the graph's topology directly, rather than as a correction applied to pairwise edges afterward, which is both more robust (nothing to "un-connect" later) and structurally simpler.

Terminal snapping — directional-cone search, not a blind radius. Each symbol's terminal has a known pin direction (recovered from the matched reference library entry — pin tip minus pin body, rotated to match the candidate's orientation). Snapping searches along that direction within a cone (`terminal_snap_cone_deg=70`) out to `terminal_directional_reach=45` px — farther than the isotropic fallback radius (`terminal_snap_radius=30` px, used only when no pin direction is known, e.g. an inferred terminal) specifically *because* restricting the search to a narrow cone instead of a full circle keeps false-positive risk low even at greater range.

Nets as the primary object, pairwise connections derived, not built directly. A schematic conductor joins N terminals, not two — one rail can touch three or four components. Building pairwise links directly from traced segments would force the pipeline to re-derive "these three are the same conductor" during dedup, and every junction merge becomes a special case. A `Net` (an equivalence class of terminals) is the primary connectivity object; the JSON's `connections` field and the delivered component graph are both *projections* of the same net data, computed at emit time — meaning they can never silently disagree with each other, because one is derived from the other, not recomputed independently.

A real bug that nearly doubled connectivity coverage — know this one cold, it's the best "attention to detail" story available. KiCad's `.kicad_sym` pin geometry: (at x y angle) is the pin's *outer* electrical connection point (where a real wire attaches), and `length` walks *inward* toward the symbol body from there. The first version of the KiCad importer had this backwards — treated (at ...) as the inner anchor and the length-extended point as the outer tip. Every terminal was consequently placed near the component body instead of at the true wire-contact point, which is *why* terminal snapping originally needed an implausibly large radius to find any connections at all, and *why* an added directional cone search made results *worse*, not better — it was searching into the component body, not toward the wire, so a narrower directional search found even less than the isotropic fallback did. Confirmed by checking the actual `R_US.kicad_sym` source: pin 1 sits at $y=3.81$, outside the zigzag body's $y=2.286$ extent, with `length` walking toward *smaller* y — i.e., into the body, confirming the convention. Fixing the two-point swap alone (no other change) moved connectivity coverage from 8/25 to 12/25 on D5 and 14/30 to 22/30 on D4 in the same run.

Duplicate-detection merge — a second real bug, caught by inspecting actual output. Two near-identical YOLO boxes (different anchors/ scales) can each independently pass YOLO's own NMS while being 1–2px apart on the same physical symbol — observed concretely on D4's MOSFET and a transistor, each boxed twice under different ids, which cascaded into an over-merged connectivity net (both near-duplicate symbols' terminals snapping to the same wire, inflating the apparent connection count). Fixed by extracting the density-localizer's existing overlap- merge logic into a shared module (`localize/merge.py`) and applying it to YOLO's output too — keeping the *higher-confidence* duplicate's class rather than discarding class info on merge (duplicates aren't always redundant noise; they can be independent votes at different confidences). Verified: D4's symbol count dropped 24→17, max pairwise IoU across all remaining symbols 0.023 (i.e., genuinely no more duplicates).

6. JSON and graph output

JSON schema — deliberately minimal, matches the brief's example exactly. `{"symbols": [{"id", "type", "coordinates", "connections"}]}` — four fields, nothing decorative. This was an explicit simplification request partway through the project (an earlier version emitted a richer object with value/confidence/terminals/rotation/nets/validation) — the richer internal representation still exists and still drives the graph construction and validation checks, it's just not dumped into the delivered JSON, since the brief's own example is the contract to match.

Graph — built bipartite first, projected for delivery. Internally: symbol nodes *and* net nodes, edges carrying terminal name/index. *Why bipartite rather than symbol-to-symbol directly:* a 5-terminal net expanded directly into a symbol-graph becomes a 10-edge clique implying pairwise wires that don't physically exist — the bipartite form is the lossless, faithful representation. Projected down to the plain component graph (nodes = symbols, edges = "shares a net") for the delivered visualization — and that projection is built *from the same trimmed JSON* the file output uses (`build_component_graph_from_json`), not recomputed independently from the richer internal graph, specifically so the two deliverables (JSON and graph) can never silently drift apart.

A layout bug worth mentioning if graph rendering comes up: `spring_layout` let connected clusters visually collapse on top of each other, hiding their own connecting edges (looked like "no connections" even when the underlying data was correct). Fixed with a custom grid-of- components layout — each connected component gets its own spring layout, then components are tiled on a grid, so clusters never overlap each other even when they'd naturally want to.

7. Data sourcing — no manually labeled data, anywhere

KiCad as the reference source. Every symbol image and terminal template traces back to KiCad's own open-source `.kicad_sym` component libraries (CC-BY-SA 4.0), parsed via a hand-built S-expression parser + rasterizer (`classify/kicad_import.py`) — not a found icon set, not crops from D4/D5. *Why:* electrical symbols are already a standardized, high-fidelity, zero-licensing-risk "ground truth" for what each class looks like — reinventing or scraping one would be strictly worse.

Multiple real style variants per class, not one canonical rendering. D4 and D5 already disagree on convention (D4 draws resistors as IEC rectangle boxes; D5 draws them as ANSI zigzags) — any real corpus will disagree even more. The library now holds both conventions for several classes (Resistor: box + zigzag; Battery: single-cell + multi-cell; Transformer: dash-core + wavy-winding; Ground: single-bar + multi-bar earth symbol; Switch: toggle + pushbutton), each sourced by fetching the actual alternate KiCad symbol and visually verifying it against real crops before trusting it — never assumed from a class name alone.

Procedural generation for parameters no fixed symbol can express

(`scripts/generate_procedural_variants.py`): battery cell count (2–6, parametrically drawn with the correct alternating long/short plate convention and "+" mark placement), resistor zigzag peak count (3–6), and a curved-bottom-plate polarized capacitor (this project's actual drawn convention, distinct from

the KiCad reference's thickness-based convention). Every shape was visually spot-checked for recognizability before use — this is generation, not invention of new conventions.

Every model touching the two evaluation drawings only for measurement, never for training. This is worth stating explicitly and precisely if asked: D4 and D5 are read *by a human* (to build the golden ground-truth JSON) and *by the pipeline* (to produce predictions to measure) — never by any training script.

8. Evaluation methodology

Why a golden set was built at all. With two unlabeled drawings, “validation” started as self-consistency checks and visual review — honest, but not falsifiable, and every subsequent accuracy claim would have been an eyeball estimate. A golden set makes every number reproducible and disprovable.

How it was built — hand verification, not automated. For each drawing: ran the current pipeline, rendered an id-only overlay (boxes + symbol id, no predicted type, so the true type had to be read from the actual schematic rather than unconsciously copying the prediction), then cross-referenced every detected id against the real drawing — zoomed crops for any ambiguous case. False positives (a detection with no real corresponding component — e.g., a non-component drawing glyph like an output bracket/arrow that D5 contains) and duplicate detections were documented and *excluded* from the golden symbols list, so `edp eval` correctly scores them as false positives rather than silently dropping them from consideration. Two real components in D4 that the pipeline missed entirely were added with estimated coordinates so recall is scored honestly, not just precision.

The `edp eval` harness (`src/edp/eval.py`). Matches predicted to golden symbols by IoU (≥ 0.5 , greedy best-IoU-first — provably optimal at this small scale), reports detection precision/recall/F1, and classification accuracy on *matched* pairs only (a detection miss is a localization problem, not a classification one — conflating them would make a bad detector look like a bad classifier). Also reports every individual confusion (`predicted=X true=Y`), every false negative, and every false positive by id, so a failure is always traceable to a specific symbol, not just a percentage.

The critical methodological finding — know this cold, it's the single best answer to “what did you learn.” Every improvement was initially measured against D4 alone (the only ground truth that existed while thresholds were being tuned). D4 alone climbed to 81.2%. Building D5's golden set the same way was the actual test of generalization — it wasn't fully there: 30.8% on the identical architecture. Root-caused, not just reported: D5's real labels are legible to a human eye but a specific font/rendering defeats Tesseract, so the OCR evidence source silently contributed nothing on D5 even on cases it should trivially resolve. This is presented honestly in every doc as *the* reason the headline number is 65.5%, not the more flattering 81.2%.

9. Measured-and-reverted — the engineering-discipline evidence

Three changes, in chronological order, each fully built, each measured, two reverted. This is the strongest evidence of judgment available — have all three numbers memorized.

#	Change	What looked good	What was actually measured	Verdict
1	21-class YOLO retrain (added Optocoupler/Potentiometer/Relay/Load classes, retrained detector)	Synthetic mAP50 >0.98 on every new class	Real D4 detection F1 0.914 → 0.686	Reverted same day
2	Roboflow “Circuit Recognition” dataset (vetted, MIT-licensed, 4/8 classes matched) mixed into YOLO training	Synthetic mAP50 0.912, in line with baseline	Real D4+D5 detection F1 0.879 → 0.174	Reverted within the hour
3	OCR designator-match confidence 0.55 → 0.75 (a plausible-sounding recalibration)	—	Zero measured change to <code>edp eval</code> output at all	Reverted — no evidence to justify keeping it

Believed root cause for #1 and #2 (stated as a hypothesis, not asserted as fact): a fixed ~3M-parameter YOLOv8n’s capacity gets spread thinner across more classes, or the training distribution shifts further from the real target domain at scale — and synthetic validation, sampled from the same distribution the model trained on, structurally cannot see either effect. Only measurement against real, held-out, hand-verified drawings can.

#3 is worth mentioning even though it’s the least dramatic — it shows the discipline isn’t selective. A change that *sounds* reasonable (“designators have been reliable all session, trust them more”) was reverted purely because it produced no measured benefit, not because it looked risky.

10. The numbers, full timeline

Milestone	Classification accuracy (D4+D5 combined)	Detection F1
Baseline (rule-based YOLO/DINOv2 priority fusion)	56.2%	0.914 (D4 only measured at this point)
+ OCR reference-designator prior	68.8% (D4 only)	unchanged
+ Geometry specialists	75.0% (D4 only)	unchanged
+ Linear probe	81.2% (D4 only)	unchanged
D5 golden set built — combined number established	58.6%	0.879
+ Wider reference library (KiCad style variants + procedural)	62.1%	unchanged
+ General OCR robustness (multi-pass merge + token-independent matching fix)	65.5%	unchanged
Session total	56.2% → 65.5%	0.879 (stable throughout)

Detection F1 never moved from its measured baseline except during the two reverted regressions in §9 — every classification-side change was scoped to leave detection untouched, which is itself a deliberate scoping choice worth naming if asked: separating “does it find the symbol” from “does it name the symbol correctly” kept each change’s blast radius small and each regression easy to attribute to a specific cause.

11. Limitations — the honest list, in priority order

1. **OCR-quality variance across drawing fonts/styles** is the largest *currently measured* source of the D4/D5 gap. Not fixable by more synthetic data — it's a rendering-specific OCR engine limitation. Next step: targeted preprocessing (contrast/sharpening tuned for text regions) or benchmarking an alternative OCR engine, gated by the same measure-before-trusting discipline as everything above.
 2. **Two confusion pairs remain partially or fully open:** BJT-vs- Potentiometer (specialist exists, sometimes still outvoted by strong prior evidence) and NPN-vs-PNP (specialist built, tested, found unreliable on real data, deliberately not shipped — see §3.6).
 3. **No connectivity ground-truth pass yet.** The golden sets verify symbol *type*, not net-level connectivity — that's the next honest measurement gap, not something already covered by the classification numbers.
 4. **Detector generalization is the one place trained-model capacity is a real, currently-unresolved constraint** — demonstrated concretely by the two reverted retrain attempts, not just asserted as a risk.
 5. **A gated VLM arbiter for residual hard cases remains unimplemented by design**, pending evidence that deterministic methods have actually plateaued — a real next step, not a rejected idea.
-

12. Rapid-fire Q&A

"Why not one big trained model end to end?" Wouldn't be debuggable or extensible the way this needs to be — a wrong answer would be untraceable to a specific cause, and adding a new symbol class would require retraining instead of a data operation.

"Why is classification accuracy your bottleneck and not detection?" Detection (0.879 F1) benefits from a genuinely narrow task — "is there a symbol-shaped blob here" transfers well from synthetic training. Classification requires fine-grained distinction (21 classes, several genuinely similar pairs) that's harder to get right without real training exposure, which is exactly the constraint described in §11.

"What's the latency?" Roughly 15–20s end to end per drawing on this laptop's GPU (RTX 3050, 4GB) after the embedding-library cache fix — dominated by OCR (multi-pass, ~4-6s) and YOLO+DINOv2 model loading, not by any single algorithmic bottleneck.

"How would you productionize this?" Reference-library and probe artifacts are already versioned build outputs, not live-computed; the main remaining production concerns are the OCR robustness gap (§11.1) and formalizing a regression-gate CI step that reruns `edp eval` on every change automatically, rather than the current discipline of doing it by hand every time (which worked, but doesn't scale to a team).

"What was hardest?" Not a specific algorithm — it was recognizing, twice, that a change with excellent synthetic numbers was actually a regression, and having the discipline to revert rather than rationalize keeping it because of the effort already invested.

13. Stress-test questions — thinking like the assessor, not the candidate

The questions above are the ones you'd volunteer. These are the ones designed to find where the story is thinner than it sounds, or where a confident answer would actually be overclaiming. Each one gets the honest answer, including the ones where the honest answer concedes a real limitation — that's a stronger position in an interview than a confident non-answer, and it's consistent with how this whole project was actually run.

"Why didn't you just point a frontier VLM at the image and ask it to describe the circuit? Claude/GPT-4V are very good at this now — wouldn't that likely beat 65%?"

This is the single most likely hard question, and it needs a precise answer, not a defensive one.

Three separable reasons, in order of how strong they are:

1. **The brief structurally requires you not to.** It lists preprocessing logic, symbol/line detection *strategy*, connectivity inference, post-processing/validation, JSON generation, and graph construction as things *you must develop* — a VLM call that returns a finished description doesn't produce any of those as inspectable stages. Even if a VLM call were dropped in as one component, all six of those surrounding stages would still need to exist and be justified independently — so "just use a VLM" doesn't actually reduce the scope of what has to be built and defended.
2. **Explainability is the explicit, named evaluation axis, not accuracy.** A VLM's answer is a finished description; there is no equivalent of `evidence_trace` (which source voted what, and why) to point at when asked "why did you decide this box is a BJT_NPN and not a Potentiometer." Every decision in this pipeline traces to a specific rule or a specific model with measurable, isolatable behavior. Losing that traceability is losing the thing being graded, even in a hypothetical world where the raw accuracy number were higher.
3. **It's a real, honest possibility that raw accuracy would win** — this should be conceded directly, not argued around. Say so plainly: "A frontier VLM would plausibly get a higher first-pass number on these two specific drawings. It would not give me a system I can add a symbol class to without touching a prompt and hoping, would not give me per-decision confidence I can fuse with other evidence, and wouldn't give me the offline, deterministic, dependency-light system the brief's own technical-expectations section is describing when it asks for pre-processing logic, a detection *strategy*, and post-processing as separate, ownable stages."

If pushed further ("but couldn't you use a VLM *and* keep the stages"): yes — and that's exactly what §3.5 already does, just for text authoring rather than runtime inference, and it's the explicitly-scoped next step (§3.6/§11: a gated VLM arbiter for residual hard cases) — not implemented because there's no evidence yet that the deterministic sources have actually plateaued, and building it before that evidence exists would be exactly the kind of unjustified complexity this project was built to avoid.

“Your golden set is 32 symbols across 2 drawings. How much can you actually conclude from a percentage-point change on a sample that small?”

Concede the math directly, then show you know what *is* still trustworthy. With 29 matched detections, **one symbol changing correctness moves the number by about 3.4 percentage points**. So a change like 62.1%→65.5% (one symbol’s worth of net movement, roughly) should not be reported as “proof the technique works in general” — it’s *consistent with* the technique helping, and specifically it’s evidence of *no regression*, which is the weaker but still real claim actually being made each time. The larger jumps are the trustworthy ones: 56.2%→65.5% combined, or 75.0%→81.2% on D4 alone from the linear probe, are multiple symbols’ worth of movement, well outside single-symbol noise.

The actual mitigation, not a dismissal of the concern: every kept change was checked against **two independently-tuned-against drawings**, not one — which is a real (if small) form of held-out validation, and it’s exactly what caught the D4-only-81.2%-vs-D5-30.8% overfitting risk in the first place. The honest limitation, stated plainly if asked “so is two drawings actually enough”: no — it’s enough to catch gross overfitting and gross regressions, not enough to certify a specific percentage as a stable estimate of true accuracy on an arbitrary new drawing. More real, hand-verified drawings is the single most valuable thing that would change this (already the answer given to “what would you do with more resources”).

“Classification accuracy is computed only on matched detections. Doesn’t that let a bad detector hide behind a good number?”

Yes, structurally — and it’s a deliberate methodological choice with a named trade-off, not an oversight, but be ready to also give the blended number so it doesn’t look like hiding behind the choice:

Why matched-only, deliberately: conflating detection and classification into one blended number makes a failure impossible to attribute — a low score could mean “the localizer never found it” or “it found it and named it wrong,” and those need completely different fixes. Reporting them separately (detection P/R/F1, classification accuracy *given* a correct detection) keeps each number diagnostic.

The blended, true end-to-end number, computed on request: of all 32 golden symbols, 29 were detected and 19 of those were also correctly classified — $19/32 \approx 59.4\%$ actually end-to-end correct, a few points below the 65.5% headline. Knowing this number cold and volunteering it unprompted is a much stronger answer than being caught not having computed it.

“How were the fusion weights (YOLO 1.2, everything else 1.0) actually chosen? Grid search, or a guess?”

Be honest: a single reasoned default, not a search. The reasoning (YOLO is the supervised, in-domain signal, so it gets a modest, not dominant, edge) is real and was checked for not causing regressions, but the specific value 1.2 was not swept against alternatives (1.0, 1.5, 2.0...) systematically. This is a genuine, named gap — the “ablation framework” discussed but not built (config-level weight toggles already support it, since `evidence_weights` is exposed exactly for this) would be the correct way to actually tune this rather than assert it. Good answer if pushed on “isn’t that unscientific”: the fusion *architecture* (weighted-sum-of-independently-abstaining-sources, replacing a fixed priority order) is the actual justified design decision, evidenced by measured improvement; the specific weight *value* is a reasonable default that hasn’t regressed anything, which is a weaker but honestly stated claim.

“The geometry specialists have hand-picked pixel thresholds — 14px minimum run length, 0.4 fill ratio, Hough param2=30. Isn’t that just as brittle as hardcoding to two drawings?”

Partially concede, then draw the real distinction. The *thresholds* were calibrated using the only real crops available (D4/D5, plus this project’s own KiCad reference renders) — so yes, there is a real, acknowledged risk they’re overfit to those specific renderings and wouldn’t transfer cleanly to a very differently-scaled or differently- styled drawing. This is *the same category of risk* as any classical CV threshold (the adaptive-binarization block size, the density thresholds in the original localizer) — calibrating a numeric constant against available examples is normal classical-CV practice, not unique brittleness, but it’s real and worth naming rather than hiding.

The distinction that *does* hold: the **rule** each specialist encodes (plate count and curvature distinguish battery/capacitor family; circle presence distinguishes BJT from potentiometer) is a general, documented drawing convention that holds for *any* schematic in this symbol family, not something specific to D4 or D5’s content — only the numeric calibration is drawing-specific, and that’s a much narrower, more honestly-scoped risk than “the whole approach only works on these two images.” If pushed further: this is exactly why the NPN/PNP specialist (§3.6) was built, measured, and *not shipped* — the same calibration process was applied and it didn’t generalize past the two reference renders it was tuned on, and the honest response was to leave it out, not force it in.

“You built a linear probe that operates on the same DINOv2 embeddings as the nearest-neighbor matcher. Isn’t that redundant — why not just replace nearest-neighbor with the probe?”

Because they were measurably wrong on different cases, not the same ones — mirroring exactly the reasoning that justified fusing YOLO and DINOv2 in the first place (non-overlapping error sets are the whole argument for fusion over replacement). The probe was kept as an *additional* vote specifically because a bad probe reading should only be able to be outvoted by a good nearest-neighbor reading, never allowed to silently override it — replacing nearest-neighbor outright would lose that safety property and bet everything on the probe generalizing better in every case, which wasn’t measured to be true, only true on average.

“Five evidence sources, multi-pass OCR, geometry specialists — isn’t this a lot of moving parts and latency for two evaluation drawings? Why not a simpler two-source system?”

Fair complexity critique, answer with the actual measured deltas rather than architecture-for-its-own-sake: each source was added *after* the previous configuration’s specific error cases were inspected and a specific source was identified as the fix — OCR closed cases DINOv2/YOLO both missed (a part number is evidence neither shape-matching approach can see), geometry closed cases where DINOv2/YOLO/OCR were all wrong together (Battery vs. Capacitor is a structural, not textual or gross-shape, distinction). Every addition is justified by a *specific, named* prior failure it fixes, not a hypothesis. On cost: total pipeline latency is ~15-20s, dominated by model loading and OCR, not by evaluating five lightweight fusion sources per candidate — the marginal cost of an additional source is small relative to what’s already paid to run YOLO+DINOv2 at all.

“Sourcing ‘multiple real style variants per class’ from KiCad — you had to look at KiCad’s library and decide which symbols looked similar enough to include. Isn’t that manual curation, i.e. hand-labeling in disguise?”

Precise distinction to draw here: it’s manual *curation of which existing, independently-authored reference source to include* — never manual *annotation of what’s in the evaluation drawings*. The symbols themselves, their classification, and their pin geometry all come from KiCad’s own published library metadata, not from a human looking at D4 or D5 and writing down “this box is a resistor.” The “no manually labeled training data” claim is specifically about never hand-annotating the *evaluation* data or *training* images pixel-by-pixel — choosing which of KiCad’s own pre-existing, pre-classified symbols to pull in is closer to picking which open dataset to use than to labeling one.

“If evaluation criteria explicitly deprioritize accuracy, why spend so much of the session’s time on OCR priors, geometry specialists, and a linear probe instead of demonstrating scalability directly?”

Two honest parts to this answer. First: the accuracy work *is* the demonstration of the architecture’s scalability property in action — every one of those additions was itself an instance of “extend via a new independent evidence source or new reference data, not a retrain,” which is the concrete proof of the “generic, scalable” claim, not a detour from it. Second, more self-critically: yes, a meaningful fraction of that time would have been better spent earlier on a formal ablation framework and the consolidated documentation, and that reallocation *did happen* — once the rubric’s explicit weighting was actually read carefully partway through, further accuracy work was consciously stopped in favor of consolidating documentation and evaluation methodology, which is itself worth mentioning as an example of course-correcting once better information was available.

“65.5% classification accuracy — isn’t that just... not very good?”

Don’t get defensive; reframe with the number that actually matters for the rubric. In isolation, no, it isn’t a high number. What’s being graded per the brief’s own stated weighting is whether it’s a *generic, scalable, well-justified* pipeline, and whether every decision along the way is defensible — and the strongest evidence for that isn’t the percentage, it’s the trail behind it: a reproducible measurement harness built before making further claims, five independently-justified improvements each with a measured before/after, and two regressions caught and reverted with real numbers instead of shipped on faith. A higher number produced by trusting synthetic validation (as both reverted experiments did) would have been a *worse* answer to this exact question, not a better one.

“Walk me through your test suite — how do you know the code itself is correct, separately from the model outputs being right?”

36 unit tests across the classification-evidence layer (fusion math, OCR designator matching including the token-independence fix, the geometry specialists against known reference crops, the embedding cache signature/invalidation logic), plus JSON-schema conformance tests. They deliberately don’t touch GPU-backed models (DINOv2, YOLO) directly — those are validated by `edp eval` against real output instead — the unit tests cover the deterministic logic surrounding the models: does the fusion formula pick the right winner given known inputs, does the designator regex correctly reject an unparseable token, does a

plate- count of 4 correctly trigger the Battery rule. If pushed on coverage gaps: the wire/connectivity stage (skeletonization, junction decomposition, net-building) has no dedicated unit tests yet beyond `test_nets.py` 's existing coverage — a real, nameable gap, not a hidden one.

“What would you do differently if you started over?”

Build the golden-truth evaluation harness *first*, before any classification improvement work, rather than after several rounds of ad-hoc D4-only tuning — the D4-vs-D5 overfitting risk would have been visible from day one instead of discovered partway through. Everything built before that point turned out fine, but it was validated later than it should have been designed to be validated.